

ADAMS SQL Reviewer

Beginner MySQL Practice Guide for Possible Panel Questions

Important clarification: Generic starter SQL examples are intentionally kept because they help you answer simple panel requests such as SELECT, WHERE, COUNT, JOIN, UPDATE, and DESCRIBE. The updates only correct table/column names and final user-role wording.

Working Local Setup	Value
System URL	http://localhost:8090/doc_ad_sys
MySQL Workbench Host	127.0.0.1
MySQL Workbench Port	3307
Database	doc_admission_db
Common local username/password	root / Root12345!
Final user roles for defense	Superadmin and Staff only
Schema reminder	Actual DB dump may still show old legacy role values, but final scope is Superadmin and Staff.

Locked ADAMS Tables and Key Attributes

Table	Correct attributes to remember
users	user_id, username, password, email, full_name, phone, is_active, created_at, last_login, role
students	student_id, user_id, student_name, last_name, first_name, middle_name, birth_date, email, category_id, remarks, phone
student_categories	category_id, category_name, sort_order
requirement_types	requirement_id, requirement_name, description
category_requirements	category_id, requirement_id
student_requirements	student_requirement_id, student_id VARCHAR(20), requirement_id, file_name, file_path, file_content, mime_type, file_size, uploaded_by, status, upload_date, uploaded_at
documents	document_id, student_id VARCHAR(20), document_type, file_path, tracking_number, status, date_submitted, remarks, student_pk
activity_log	log_id, user_id, username, action, module, description, log_time
print_logs	log_id, ref_no, student_id, document_type, printed_by, printed_by_name, printed_at, campus_code
password_reset_tokens	id, user_id, token, expires_at, used, created_at
archived_students	archive_id, student_id, student_name, last_name, first_name, middle_name, email, phone, birth_date, category_id, remarks, archived_at, archived_by

SQL Keyword Cheat Sheet

Keyword	Meaning	Example Use
SELECT	Displays/retrieves data	SELECT * FROM students;
FROM	Identifies the table source	FROM users
WHERE	Filters rows	WHERE role = 'staff'
LIKE	Searches text pattern	LIKE '%Santos%'
ORDER BY	Sorts results	ORDER BY last_name ASC
LIMIT	Limits number of rows	LIMIT 10
COUNT	Counts rows	COUNT(*)
GROUP BY	Groups records for summary	GROUP BY category_id
HAVING	Filters grouped results	HAVING COUNT(*) > 1
JOIN	Combines related tables	students JOIN student_categories
LEFT JOIN	Shows all left-table rows even without	students LEFT JOIN documents

Keyword	Meaning	Example Use
	matches	
INSERT	Adds new record	INSERT INTO users ...
UPDATE	Edits existing record	UPDATE users SET ...
DELETE	Removes records	DELETE FROM users WHERE ...
CREATE	Creates database/table	CREATE TABLE ...
ALTER	Modifies table structure	ALTER TABLE users ADD COLUMN ...

1. Quick Start and Safe Checking

Connect to local MySQL in Docker

```
docker exec -it adams-mysql mysql -uroot -p'Root12345!' doc_admission_db
```

Helpful note: Use this only if your local Docker setup uses the same container name and password.

Use the ADAMS database

```
USE doc_admission_db;
```

Helpful note: This selects the database before running SQL queries.

Show all tables

```
SHOW TABLES;
```

Helpful note: Use this first to confirm the database was imported.

Check table structure

```
DESCRIBE students;
DESCRIBE student_requirements;
DESCRIBE documents;
```

Helpful note: Use DESCRIBE if the panel asks about columns or if a query gives an unknown column error.

2. Basic SELECT Queries - Generic Starter Practice

Display all users

```
SELECT *
FROM users;
```

Helpful note: SELECT * is okay for checking the table quickly. For reports, select only needed columns.

Display all students

```
SELECT *  
FROM students;
```

Helpful note: This shows all applicant/student records.

Display selected user columns only

```
SELECT user_id, username, full_name, role, is_active  
FROM users;
```

Helpful note: This is cleaner than SELECT *.

Display only Superadmin and Staff users

```
SELECT user_id, username, full_name, role  
FROM users  
WHERE role IN ('superadmin', 'staff')  
ORDER BY role, username;
```

Helpful note: Use this because final ADAMS user roles are Superadmin and Staff only.

Search student by last name

```
SELECT *  
FROM students  
WHERE last_name LIKE '%Santos%';
```

Helpful note: LIKE is useful for search bars. Percent sign means any characters before or after.

Sort students alphabetically

```
SELECT student_id, last_name, first_name, email  
FROM students  
ORDER BY last_name ASC, first_name ASC;
```

Helpful note: ASC means A to Z. DESC means Z to A.

Show latest 10 activity logs

```
SELECT *  
FROM activity_log  
ORDER BY log_time DESC  
LIMIT 10;
```

Helpful note: Correct ADAMS column is log_time, not created_at.

3. Counting and Summary Queries

Count total students

```
SELECT COUNT(*) AS total_students
FROM students;
```

Helpful note: Useful for dashboard totals.

Count total users by final role

```
SELECT role, COUNT(*) AS total_users
FROM users
WHERE role IN ('superadmin', 'staff')
GROUP BY role;
```

Helpful note: Shows total Superadmin and Staff users.

Count students per category

```
SELECT c.category_name, COUNT(s.student_id) AS total_students
FROM student_categories c
LEFT JOIN students s
    ON s.category_id = c.category_id
GROUP BY c.category_id, c.category_name
ORDER BY c.sort_order, c.category_name;
```

Helpful note: Uses LEFT JOIN so categories with zero students can still appear.

Count submitted document records per student

```
SELECT s.student_id, s.student_name, COUNT(d.document_id) AS total_documents
FROM students s
LEFT JOIN documents d
    ON d.student_pk = s.student_id
GROUP BY s.student_id, s.student_name
ORDER BY s.student_name;
```

Helpful note: Correct link is documents.student_pk to students.student_id.

Find duplicate usernames

```
SELECT username, COUNT(*) AS total
FROM users
GROUP BY username
HAVING COUNT(*) > 1;
```

Helpful note: HAVING filters grouped results.

4. JOIN Queries - Pulling Related ADAMS Data

Show students with their category

```
SELECT s.student_id, s.student_name, s.email, c.category_name
FROM students s
LEFT JOIN student_categories c
    ON c.category_id = s.category_id
ORDER BY s.student_name;
```

Helpful note: This is a common defense/demo query.

Show requirements required per category

```
SELECT c.category_name, r.requirement_name, r.description
FROM category_requirements cr
JOIN student_categories c
    ON c.category_id = cr.category_id
JOIN requirement_types r
    ON r.requirement_id = cr.requirement_id
ORDER BY c.category_name, r.requirement_name;
```

Helpful note: Uses the bridge table category_requirements.

Show student requirements with requirement names

```
SELECT s.student_id, s.student_name, rt.requirement_name, sr.status, sr.file_name, sr.uploaded_at
FROM students s
JOIN student_requirements sr
    ON sr.student_id = CAST(s.student_id AS CHAR)
JOIN requirement_types rt
    ON rt.requirement_id = sr.requirement_id
ORDER BY s.student_name, rt.requirement_name;
```

Helpful note: Correct field is requirement_id, not requirement_type_id.

Show students with submitted documents

```
SELECT s.student_id, s.student_name, d.document_id, d.document_type, d.file_path, d.status,
d.date_submitted
FROM students s
LEFT JOIN documents d
    ON d.student_pk = s.student_id
ORDER BY s.student_name, d.date_submitted DESC;
```

Helpful note: Correct document date field is date_submitted.

Show students with no document record

```
SELECT s.student_id, s.student_name
FROM students s
LEFT JOIN documents d
      ON d.student_pk = s.student_id
WHERE d.document_id IS NULL;
```

Helpful note: This is a missing-record query.

Show print logs with student and user

```
SELECT pl.ref_no, s.student_name, pl.document_type, u.full_name AS printed_by, pl.printed_at
FROM print_logs pl
LEFT JOIN students s
      ON s.student_id = pl.student_id
LEFT JOIN users u
      ON u.user_id = pl.printed_by
ORDER BY pl.printed_at DESC;
```

Helpful note: Correct print log date field is printed_at.

5. INSERT, UPDATE, DELETE - Practice with Safety

Insert a new Staff user

```
INSERT INTO users
(username, password, email, full_name, phone, role, is_active)
VALUES
('staff1', 'staff123', 'staff1@example.com', 'Staff One', '', 'staff', 1);
```

Helpful note: Use staff or superadmin only for final role explanation.

Update user email

```
UPDATE users
SET email = 'newemail@example.com'
WHERE username = 'staff1';
```

Helpful note: Always use WHERE in UPDATE.

Deactivate a user instead of deleting

```
UPDATE users
SET is_active = 0
WHERE username = 'staff1';
```

Helpful note: Deactivation is safer than deleting historical user records.

Delete a specific user

```
DELETE FROM users
WHERE username = 'staff1';
```

Helpful note: Before DELETE, run SELECT with the same WHERE condition.

Safety check before UPDATE or DELETE

```
SELECT *
FROM users
WHERE username = 'staff1';
```

Helpful note: If SELECT returns the correct row, then run UPDATE or DELETE.

6. Table Structure Practice

Create a simple sample table

```
CREATE TABLE sample_users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100),
    role VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Helpful note: This is generic practice for DDL, not part of final ADAMS schema.

Add a new sample column

```
ALTER TABLE sample_users
ADD COLUMN phone VARCHAR(20);
```

Helpful note: ALTER TABLE changes structure.

Modify a sample column size

```
ALTER TABLE sample_users
MODIFY COLUMN email VARCHAR(150);
```

Helpful note: MODIFY COLUMN changes data type or size.

Drop/remove a sample column

```
ALTER TABLE sample_users
DROP COLUMN phone;
```

Helpful note: DROP COLUMN permanently removes the column and its data.

Foreign key example using actual ADAMS relationship

```
ALTER TABLE documents
ADD CONSTRAINT fk_documents_student_pk_example
FOREIGN KEY (student_pk)
REFERENCES students(student_id);
```

Helpful note: Use student_pk for documents table FK example, not documents.student_id.

7. Advanced ADAMS Query Patterns

Missing or pending requirements

```
SELECT s.student_id, s.student_name, c.category_name, r.requirement_name,
       COALESCE(sr.status, 'MISSING') AS status_result
FROM students s
JOIN student_categories c
  ON c.category_id = s.category_id
JOIN category_requirements cr
  ON cr.category_id = s.category_id
JOIN requirement_types r
  ON r.requirement_id = cr.requirement_id
LEFT JOIN student_requirements sr
  ON sr.student_id = CAST(s.student_id AS CHAR)
 AND sr.requirement_id = r.requirement_id
WHERE sr.student_requirement_id IS NULL
     OR sr.status IS NULL
     OR sr.status IN ('PENDING', 'INSUFFICIENT', 'REJECTED')
ORDER BY s.student_name, r.requirement_name;
```

Helpful note: This compares required documents against actual submissions.

Requirement completion summary per student

```
SELECT s.student_id, s.student_name, c.category_name,
       COUNT(cr.requirement_id) AS required_count,
       COUNT(sr.student_requirement_id) AS recorded_count,
       SUM(CASE WHEN sr.file_name IS NOT NULL THEN 1 ELSE 0 END) AS uploaded_count,
       SUM(CASE WHEN sr.status IN ('SUBMITTED', 'VERIFIED', 'APPROVED') THEN 1 ELSE 0 END) AS
completed_count,
       COUNT(cr.requirement_id) - COUNT(sr.student_requirement_id) AS missing_count
FROM students s
JOIN student_categories c
  ON c.category_id = s.category_id
JOIN category_requirements cr
  ON cr.category_id = c.category_id
LEFT JOIN student_requirements sr
  ON sr.student_id = CAST(s.student_id AS CHAR)
```

Prepared for ADAMS review and oral defense practice only

```

AND sr.requirement_id = cr.requirement_id
GROUP BY s.student_id, s.student_name, c.category_name
ORDER BY missing_count DESC, s.student_name;

```

Helpful note: This is good for dashboard/report explanation.

Latest submitted document per student

```

SELECT d.*
FROM documents d
INNER JOIN (
    SELECT student_pk, MAX(date_submitted) AS latest_submission
    FROM documents
    GROUP BY student_pk
) latest
ON d.student_pk = latest.student_pk
AND d.date_submitted = latest.latest_submission;

```

Helpful note: Uses date_submitted from documents.

Pending requirement aging report

```

WITH requirement_status AS (
    SELECT s.student_id, s.student_name, r.requirement_name, sr.status, sr.uploaded_at,
        ROW_NUMBER() OVER (
            PARTITION BY s.student_id, r.requirement_id
            ORDER BY COALESCE(sr.uploaded_at, '1900-01-01') DESC
        ) AS rn
    FROM students s
    JOIN category_requirements cr
    ON cr.category_id = s.category_id
    JOIN requirement_types r
    ON r.requirement_id = cr.requirement_id
    LEFT JOIN student_requirements sr
    ON sr.student_id = CAST(s.student_id AS CHAR)
    AND sr.requirement_id = r.requirement_id
)
SELECT student_id, student_name, requirement_name,
    COALESCE(status, 'MISSING') AS status_result,
    uploaded_at,
    CASE WHEN uploaded_at IS NULL THEN NULL
        ELSE DATEDIFF(CURDATE(), DATE(uploaded_at))
    END AS days_since_upload
FROM requirement_status
WHERE rn = 1
    AND (status IS NULL OR status = 'PENDING')
ORDER BY student_name, requirement_name;

```

Helpful note: Uses CTE and ROW_NUMBER for latest status checking.

Possible Oral/Exam Questions and Short Answers

Question	Simple Answer
What is SELECT used for?	To retrieve/display records from one or more tables.
What is WHERE used for?	To filter rows based on a condition.

Question	Simple Answer
What is LIKE used for?	To search text using a pattern such as %keyword%.
What is ORDER BY used for?	To sort query results ascending or descending.
What is GROUP BY used for?	To group records and calculate summaries like COUNT, SUM, or AVG.
Difference between WHERE and HAVING?	WHERE filters rows before grouping; HAVING filters grouped results.
Difference between INNER JOIN and LEFT JOIN?	INNER JOIN returns only matching records; LEFT JOIN returns all records from the left table even without matches.
Why use primary key?	To uniquely identify each row in a table.
Why use foreign key?	To connect related tables and maintain data consistency.
Why avoid SELECT * in reports?	It can show unnecessary columns and can be less efficient on large tables.
Why should UPDATE/DELETE have WHERE?	Without WHERE, all records may be changed or deleted.
What should you do if there is an unknown column error?	Run DESCRIBE table_name; then correct the column name.

Common Errors and Quick Fixes

Error / Problem	Meaning	Fix
Unknown column	Column name does not exist in that table.	Run DESCRIBE table_name; and adjust the query.
Table does not exist	Wrong database or incomplete import.	Run SHOW TABLES; and check USE doc_admission_db;
Access denied	Wrong username/password.	Use your local MySQL credentials.
Cannot connect to MySQL	Container/server may not be running.	Start the MySQL container/server.
Empty dashboard	No records yet or imported data incomplete.	Check students and related tables.
Invalid login	Wrong user record or app connects to wrong database.	Check users table and database connection file.

Practice Checklist

- Practice SELECT, WHERE, LIKE, ORDER BY, and LIMIT first.
- Practice COUNT, GROUP BY, and HAVING next.
- Practice JOIN queries after you understand primary keys and foreign keys.
- Always run DESCRIBE before writing JOIN queries if you are not sure of column names.
- Before UPDATE or DELETE, test with SELECT first.
- Explain each query in your own words: what table, what condition, and what output.